



IBM Developer
SKILLS NETWORK

Winning Space Race with Data Science

Mario Arturo Hernandez Cervantes
April 1st, 2026



Outline

- Executive Summary
- Introduction
- Methodology
- Results
- Conclusion
- Appendix

Executive Summary

- Summary of methodologies:

- Data Collection
- Data Wrangling
- Web Scraping
- Exploratory Data Analysis with Data Visualization
- Exploratory Data Analysis with SQL
- Building an Interactive Map with Folium
- Dashboard with Plotly Dash
- Machine Learning
 - Logistic Regression
 - Support Vector Machine
 - Decision Tree
 - K-Nearest Neighbor

- Summary of all results:

- Exploratory Data Analysis Results
- Interactive Analytics Results
- Predictive Machine Learning Results

Introduction

- Project background and context

For this project, we will predict if the Falcon 9 first stage will land successfully. SpaceX advertises Falcon 9 rocket launches on its website with a cost of 62 million dollars; other providers cost upward of 165 million dollars each, much of the savings is because SpaceX can reuse the first stage. Therefore, if we can determine if the first stage will land, we can determine the cost of a launch. This information can be used if an alternate company wants to bid against SpaceX for a rocket launch. Most unsuccessful landings are planned and Space X performs a controlled landing in the oceans. In order to achieve this purpose, we will collect data, make wrangling data, exploratory analysis and finally apply some Machine Learning techniques.

- Problems you want to find answers

Some of the problems we want to solve are the following:

- What launch sites are most successful?
- What kind of rockets are most successful when landing and the types of relationships and variables that determine such success.

Section 1

Methodology

Methodology

Executive Summary

- Data collection methodology:
 - Request to the SpaceX API
 - Web scraping Falcon 9 and Falcon Heavy Launches Records from Wikipedia
- Perform data wrangling
 - Landing outcome label obtained by identifying and summarizing relevant outcome data.
- Perform exploratory data analysis (EDA) using visualization and SQL
- Perform interactive visual analytics using Folium and Plotly Dash
- Perform predictive analysis using classification models
 - Splitting data into train and test set, using different algorithms, choosing the best one.

Data Collection

- Request to the SpaceX API

- Start requesting rocket launch data and check response from SpaceX API:

- `spacex_url=https://api.spacexdata.com/v4/launches/past`

- `response=requests.get(static_json_url)`

- Converting response to a .json file:

- `data = pd.json_normalize(response.json())`

- Apply local functions to clean data:

- `getBoosterVersion`

- `getLaunchSite`

- `getPayloadData`

- `getCoreData`

Data Collection

- Assign list to a dictionary and then to a dataframe:

```
launch_dict = {'FlightNumber': list(data['flight_number']),
               'Date': list(data['date']),
               'BoosterVersion':BoosterVersion,
               'PayloadMass':PayloadMass,
               'Orbit':Orbit,
               'LaunchSite':LaunchSite,
               'Outcome':Outcome,
               'Flights':Flights,
               'GridFins':GridFins,
               'Reused':Reused,
               'Legs':Legs,
               'LandingPad':LandingPad,
               'Block':Block,
               'ReusedCount':ReusedCount,
               'Serial':Serial,
               'Longitude': Longitude,
               'Latitude': Latitude}
```

```
data = pd.DataFrame(launch_dict)
```


Data Collection

- Filter dataframe to include only Falcon 9 launches and export to .csv file:

```
data_falcon9 = data[data.BoosterVersion == 'Falcon 9']  
data_falcon9.to_csv('dataset_lab.csv', index=False)
```

- Web scraping Falcon 9 and Falcon Heavy Launches Records from Wikipedia

- Getting response from HTML:

```
response = requests.get(static_url)
```

- Creating BeautifulSoup object:

```
soup = BeautifulSoup(response, 'html.parser')
```

- Extract all column/variable names from the HTML table header:

```
html_tables = soup.find_all('table')
```

Data Collection

- Getting column names:

```
column_names = []
temporal = soup.find_all('th')
for x in range(len(temporal)):
    try:
        name = extract_column_from_header(temporal[x])
        if (name is not None and len(name) > 0):
            column_names.append(name)
    except:
        pass
```

Data Collection

- Creating a dictionary:

```
launch_dict= dict.fromkeys(column_names)
```

```
# Remove an irrelevant column
```

```
del launch_dict['Date and time ( )']
```

```
launch_dict['Flight No.'] = []
```

```
launch_dict['Launch site'] = []
```

```
launch_dict['Payload'] = []
```

```
launch_dict['Payload mass'] = []
```

```
launch_dict['Orbit'] = []
```

```
launch_dict['Customer'] = []
```

```
launch_dict['Launch outcome'] = []
```

```
# Added some new columns
```

```
launch_dict['Version Booster']=[]
```

```
launch_dict['Booster landing']=[]
```

```
launch_dict['Date']=[]
```

```
launch_dict['Time']=[]
```

Data Collection

- Extracting data from tables:

```
extracted_row = 0
#Extract each table
for table_number,table in enumerate(soup.find_all('table',"wikitable plainrowheaders collapsible")):
    # get table row
    for rows in table.find_all("tr"):
        #check to see if first table heading is as number corresponding to launch a number
        if rows.th:
            if rows.th.string:
                flight_number=rows.th.string.strip()
                flag=flight_number.isdigit()
            else:
                flag=False
        #get table element
        row=rows.find_all('td')
        ...
```

Data Collection

- Appending data to keys:

```
headings = []
for key, values in dict(launch_dict).items():
    if key not in headings:
        headings.append(key)
    if values is None:
        del launch_dict[key]
...
```

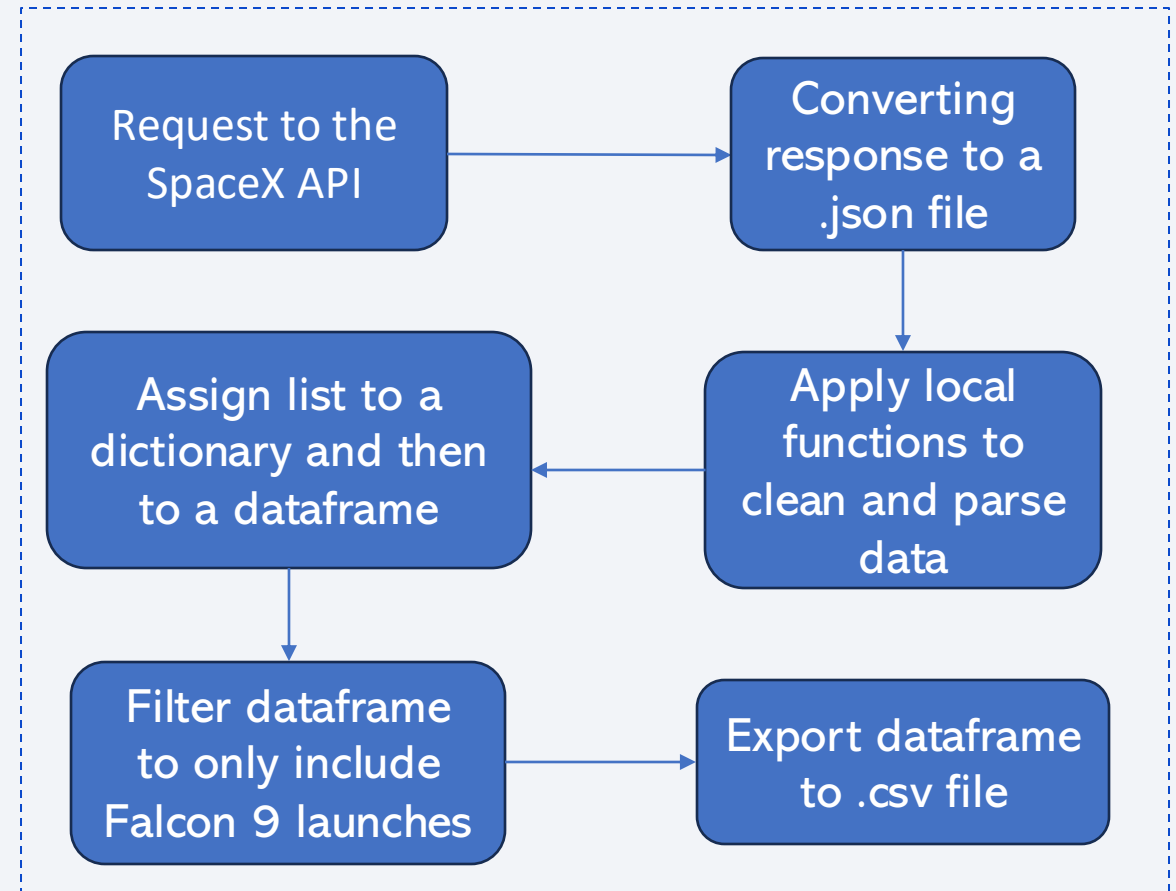
- Converting dictionary to dataframe and then to a .csv file:

```
df = pd.DataFrame.from_dict(launch_dict)
df.to_csv('spacex_web_scraped_lab.csv', index=False)
```


Data Collection – SpaceX API

- Flowchart of SpaceX REST calls:
- GitHub URL of the completed SpaceX API calls notebook:

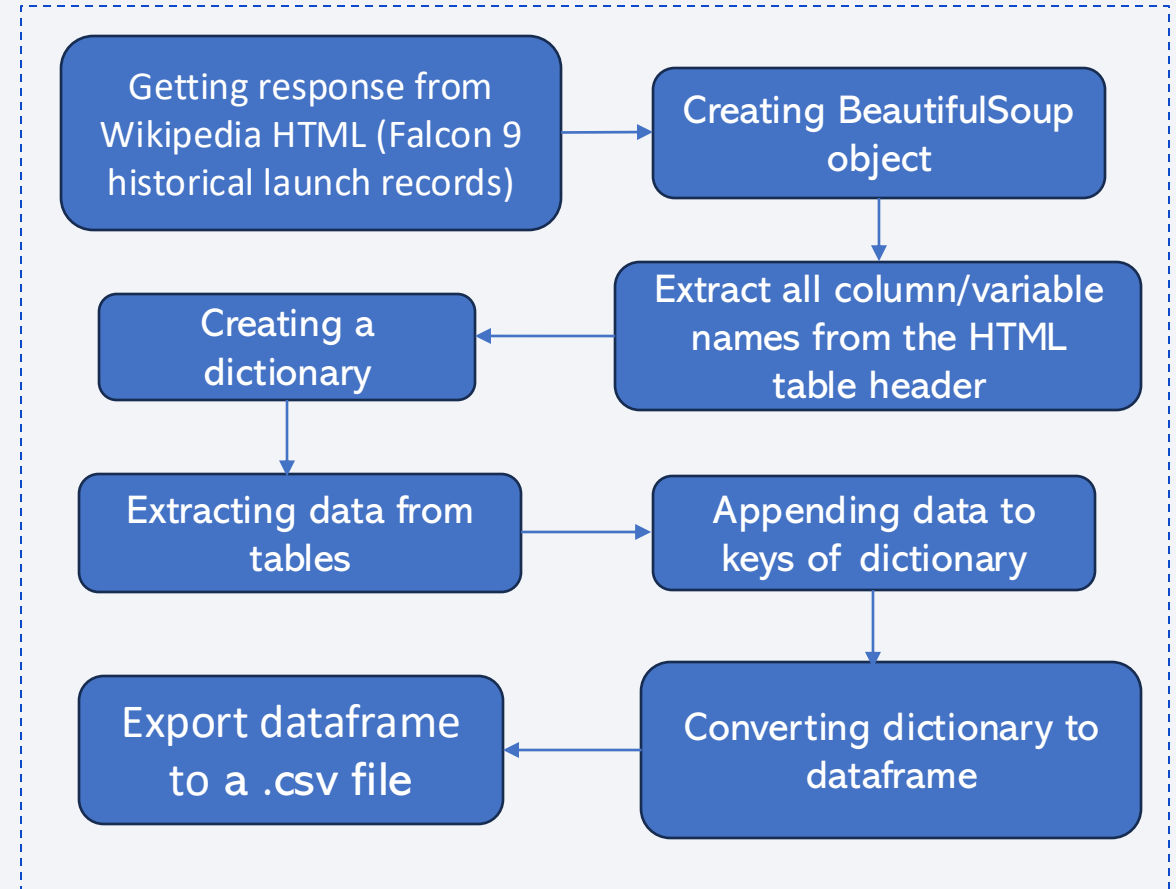
https://github.com/marioahc21/IBM_Data_Science/blob/main/jupyter-labs-spacex-data-collection-api_SOLVED.ipynb



Data Collection - Scraping

- Flowchart of Web Scraping
- GitHub URL of the completed web scraping notebook:

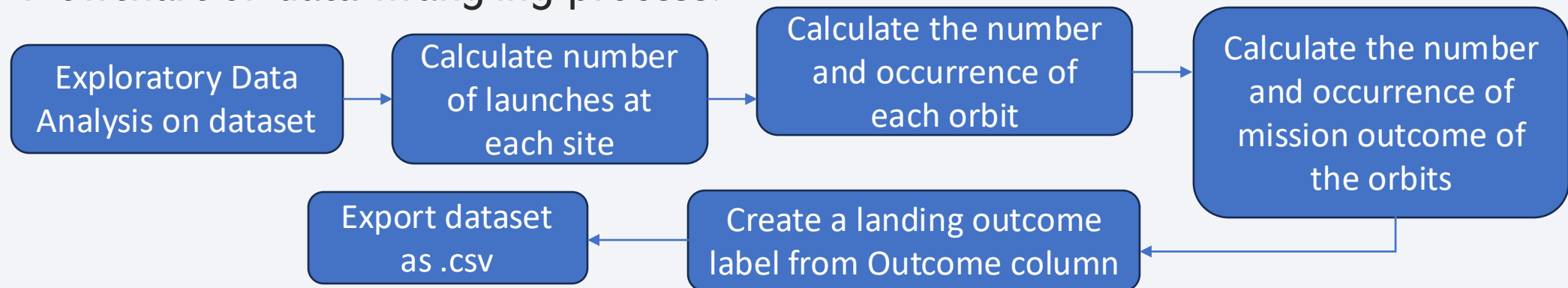
https://github.com/marioahc21/IBM_Data_Science/blob/main/jupyter-labs-webscraping_SOLVED.ipynb



Data Wrangling

We will perform some Exploratory Data Analysis (EDA) to find some patterns in the data and determine what would be the label for training supervised models. In the data set, there are several different cases where the booster did not land successfully. Sometimes a landing was attempted but failed due to an accident; for example, 'True Ocean' means the mission outcome was successfully landed to a specific region of the ocean while 'False Ocean' means the mission outcome was unsuccessfully landed to a specific region of the ocean. 'True RTLS' means the mission outcome was successfully landed to a ground pad. 'False RTLS' means the mission outcome was unsuccessfully landed to a ground pad. 'True ASDS' means the mission outcome was successfully landed on a drone ship. 'False ASDS' means the mission outcome was unsuccessfully landed on a drone ship. For this project we will mainly convert those outcomes into Training Labels with '1' means the booster successfully landed and '0' means it was unsuccessful.

- Flowchart of data wrangling process:



- GitHub URL of the completed data wrangling:

https://github.com/marioahc21/IBM_Data_Science/blob/main/labs-jupyter-spacex-Data%20wrangling_SOLVED.ipynb

EDA with Data Visualization

- Charts that were plotted and why:

- Flight Number vs Payload Mass (kg)
- Flight Number vs Launch Site
- Payload Mass (kg) vs Launch Site
- Orbit vs Class (Success Rate (%))
- Flight Number vs Orbit
- Payload Mass (kg) vs Orbit
- Year vs Success Rate

- Scatter plots show correlation between two or more variables.
- Bar diagrams help to understand sets of data between different groups at a glance.
- Line graphs display changes in direction, trends and relationships between two continuous variables.

- GitHub URL of the completed EDA with data visualization notebook:

https://github.com/marioahc21/IBM_Data_Science/blob/main/edadataviz_SOLVED.ipynb

EDA with SQL

- Summarizing the SQL queries performed
 - Display the names of the unique launch sites in the space mission
 - Display 5 records where launch sites begin with the string 'CCA'
 - Display the total payload mass carried by boosters launched by NASA (CRS)
 - Display average payload mass carried by booster version F9 v1.1
 - List the date when the first successful landing outcome in ground pad was achieved
 - List the names of the boosters which have success in drone ship and have payload mass greater than 4000 but less than 6000
 - List the total number of successful and failure mission outcomes
 - List all the booster versions that have carried the maximum payload mass, using a subquery with a suitable aggregate function.
 - List the records which will display the month names, failure landing outcomes in drone ship ,booster versions, launch site for the months in year 2015.
 - Rank the count of landing outcomes (such as Failure (drone ship) or Success (ground pad)) between the date 2010-06-04 and 2017-03-20, in descending order.
- GitHub URL of the completed EDA with SQL notebook:

https://github.com/marioahc21/IBM_Data_Science/blob/main/jupyter-labs-eda-sql-coursera_sqllite_SOLVED.ipynb

Build an Interactive Map with Folium

- Summarizing what map objects such as markers, circles, lines, etc. we created and added to a folium map
 - First, we added each site's location on a map using site's latitude and longitude coordinates from the file 'spacex_launch_geo.csv'.
 - We created a folium 'Map' object, with an initial center location to be NASA Johnson Space Center at Houston, Texas.
 - We used 'folium.Circle' to add a highlighted circle area with a text label on a specific coordinate using 'folium.map.Marker'. For example: a blue circle at NASA Johnson Space Center's coordinate with a popup label showing its name.
 - `circle = folium.Circle(nasa_coordinate, radius=1000, color='#d35400', fill=True).add_child(folium.Popup('NASA Johnson Space Center'))`
 - `marker = folium.map.Marker(...`
 - We used marker clusters because are a good way to simplify a map containing many markers having the same coordinate.
 - `marker_cluster = MarkerCluster()`
 - We customize the Marker's icon property to indicate if this launch was succeeded or failed:
 - `icon = folium.Icon(color='white', icon_color=record['marker_color'])`
 - Lines were drawn on the map to measure distance to landmarks. For example, we created a 'folium.PolyLine' object using the coastline coordinates and launch site coordinate to draw a PolyLine between a launch site to the selected coastline point:
 - `lines=folium.PolyLine(locations=[coast_coordinates, launch_site_coordinates], weight=1)`
- GitHub URL of the completed interactive map with Folium map:

https://github.com/marioahc21/IBM_Data_Science/blob/main/lab_jupyter_launch_site_location_SOLVED.ipynb

Build a Dashboard with Plotly Dash

- Summarizing what plots/graphs and interactions we have added to a dashboard
 - Graphs:
 - Pie Charts showing the total/proportional launches by certain sites (CCAFS LC-40, VAFB SLC-4E, KSC LC-39A, CCAFS SLC-40) or all sites at once. This allow us to display information about successful and unsuccessful landings on those sites.
 - Scatter chart to show the correlation between Payload Mass (kg) and Launch Success for the different Booster Versions. Minimum and maximum values can be determined.
 - A slider to select payload range between 0 and 10000 (kg).
- GitHub URL of the completed Plotly Dash lab:

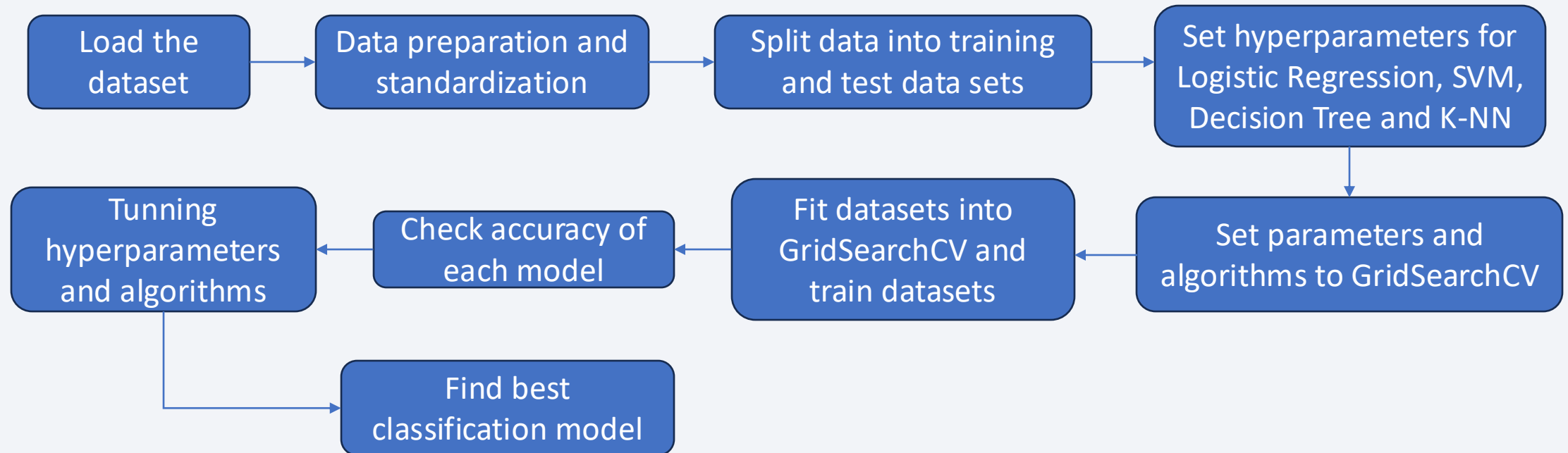
https://github.com/marioahc21/IBM_Data_Science/blob/main/spacex-dash-app.py

Predictive Analysis (Classification)

- Building Model
 - Load our dataset into NumPy and Pandas
 - Standardize the data
 - Split the data into training and test data sets
 - Check the number of test samples
 - Find best hyperparameter for Logistic Regression, Support Vector Machine, Classification Trees and K-Nearest Neighbors
 - Set parameters and algorithms to GridSearchCV
 - Fit our datasets into the GridSearchCV objects and train our dataset
- Evaluating Model
 - Check accuracy for each model by using the method score and plotting the confusion matrix
- Improving Model
 - Tuning hyperparameters for each type of algorithms
 - Algorithm tuning
- Finding the Best Performing Classification Model
 - Final report of best accuracy for all algorithms

Predictive Analysis (Classification)

- Flowchart of Predictive Analysis (Classification)



- GitHub URL of the completed predictive analysis lab:

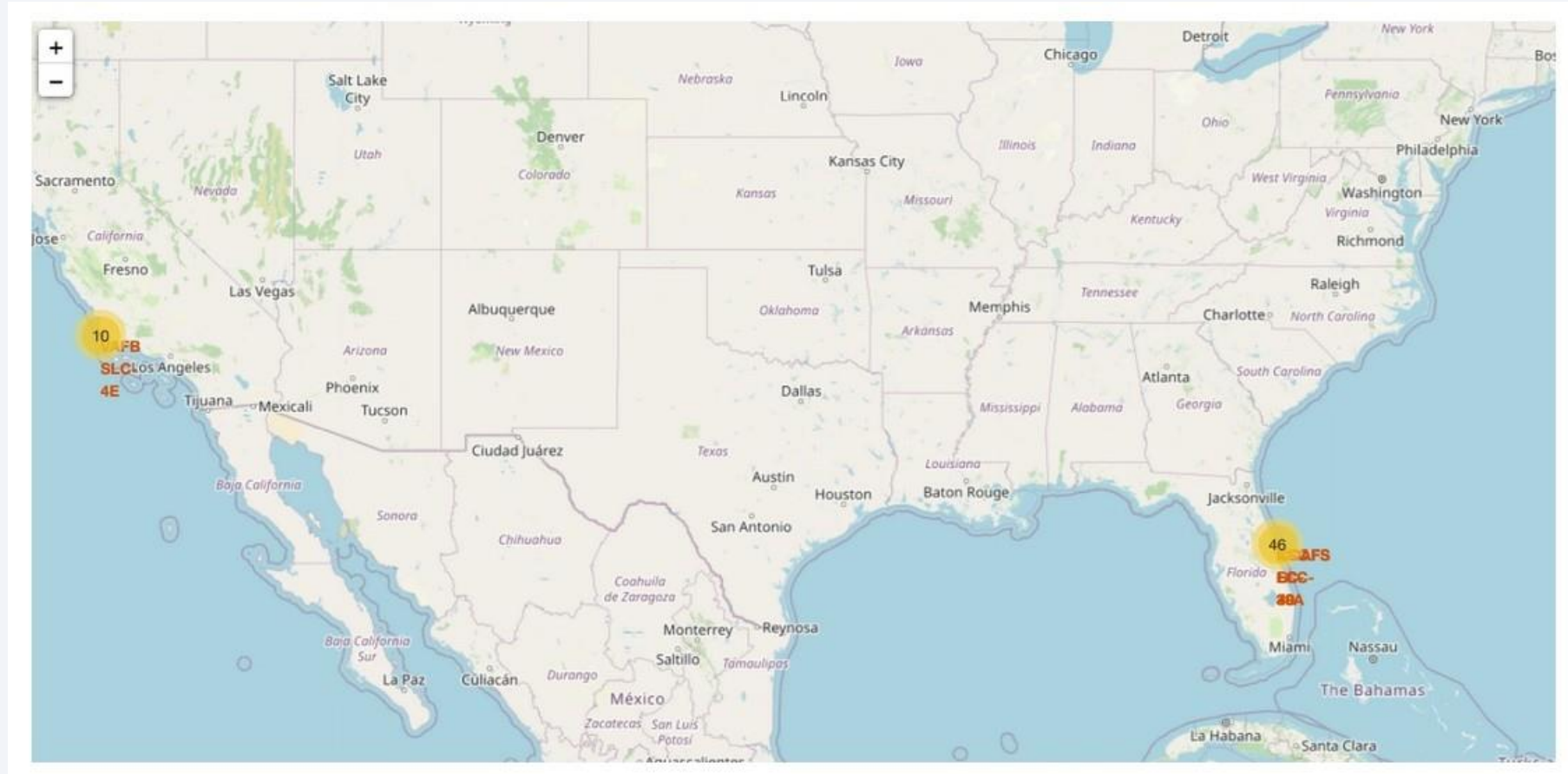
https://github.com/marioahc21/IBM_Data_Science/blob/main/SpaceX_Machine%20Learning%20Prediction_Part_5_SOLVED.ipynb

Results

- Exploratory data analysis results
 - Site CCAFS SLC-40 has the most successful landings.
 - Sites KSC LC 39A and CCAFS SLC-40 have launched the heaviest loads.
 - Observing Payload Mass vs. Launch Site scatter point chart we found for the VAFB SLC-4E launch site there are no rockets launches for heavy payload mass greater than 10000.
 - With heavy payloads the successful landing or positive landing rate are more for orbits PO, LEO, VLEO and ISS. However, for GTO, it's difficult to distinguish between successful and unsuccessful landings as both outcomes are present.
 - The success rate since 2013 kept increasing till 2020.

Results

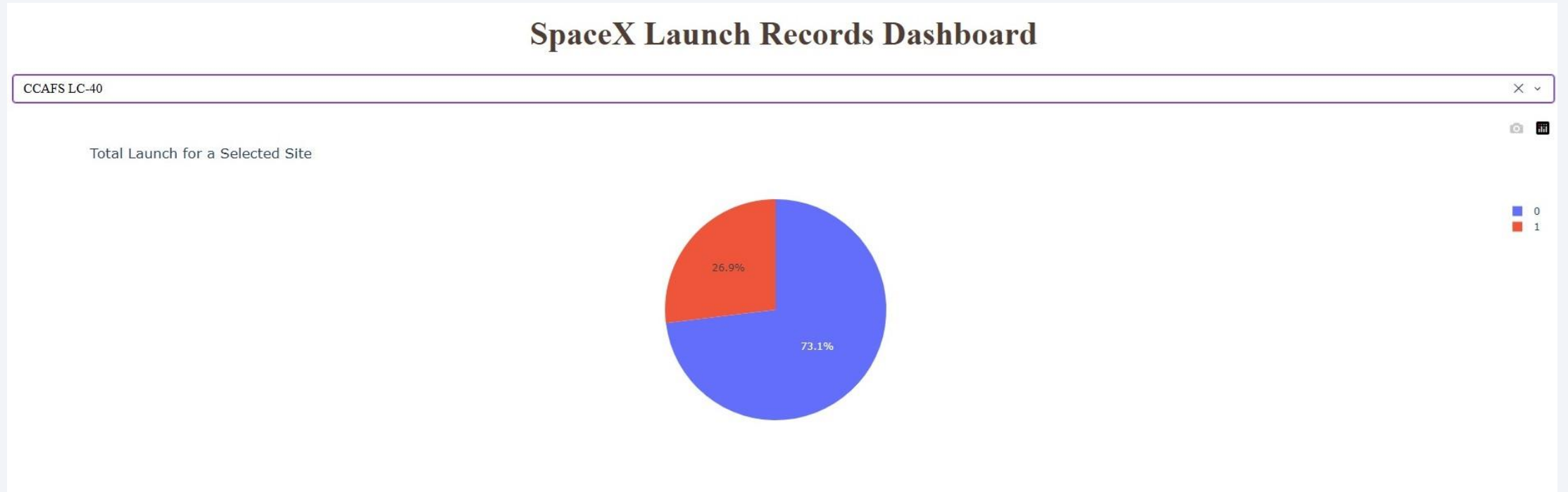
- Interactive analytics demo in screenshots



Results



Results



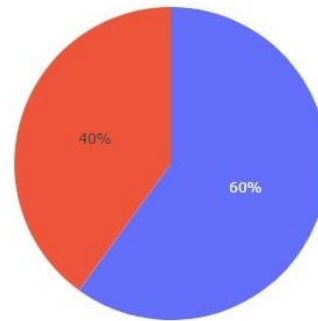
Results

SpaceX Launch Records Dashboard

VAFB SLC-4E

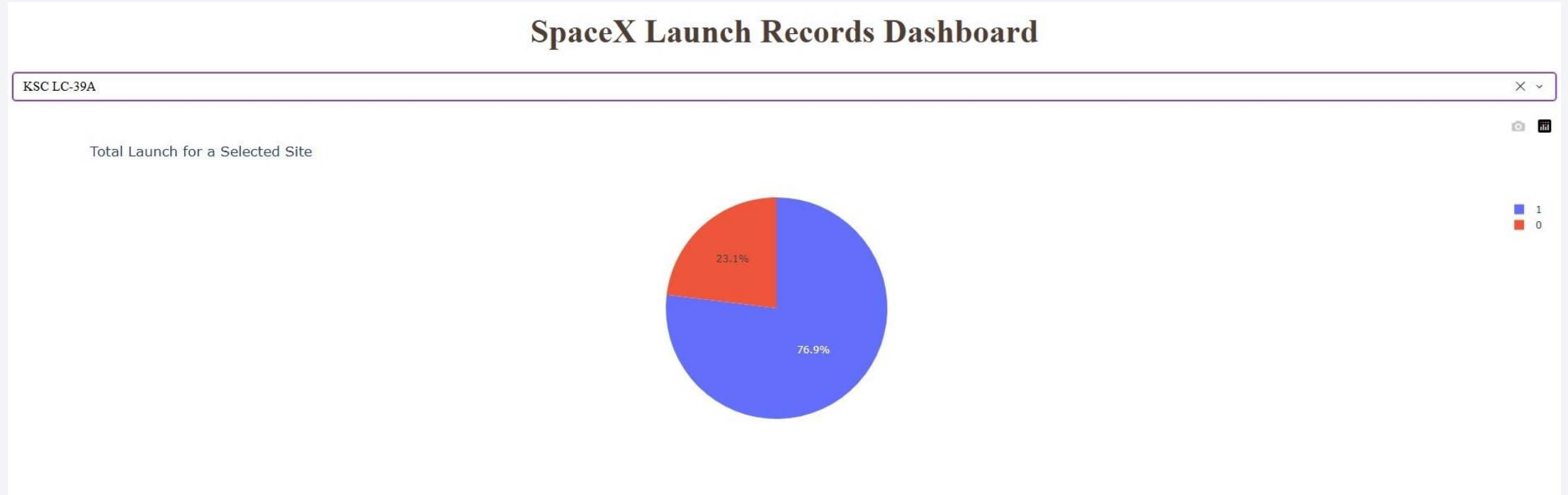


Total Launch for a Selected Site



0
1

Results



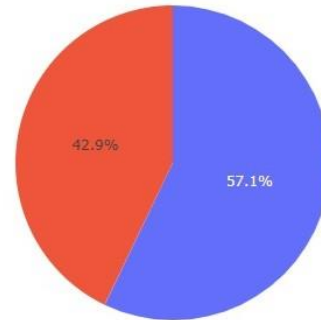
Results

SpaceX Launch Records Dashboard

CCAFS SLC-40

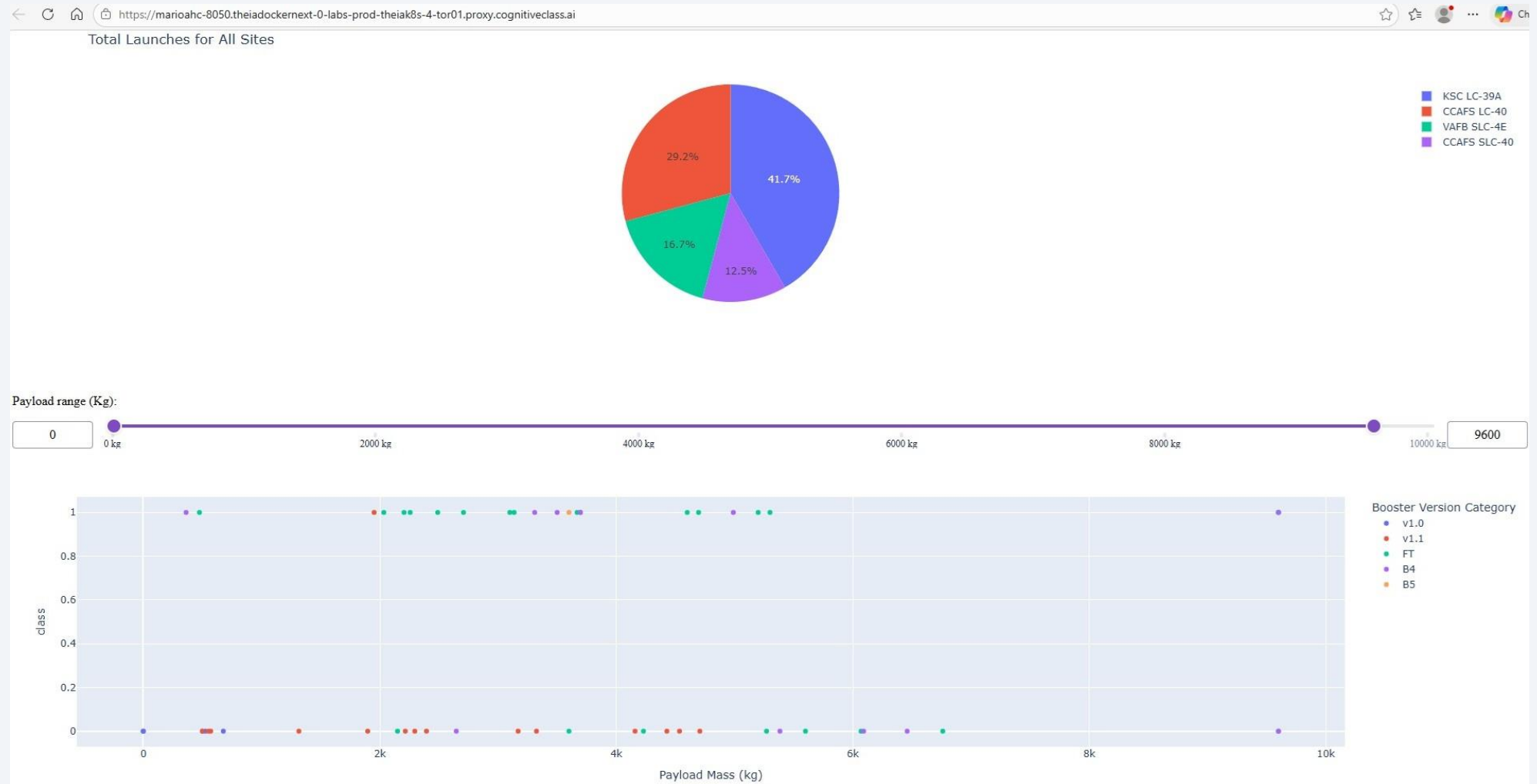


Total Launch for a Selected Site



■ 0
■ 1

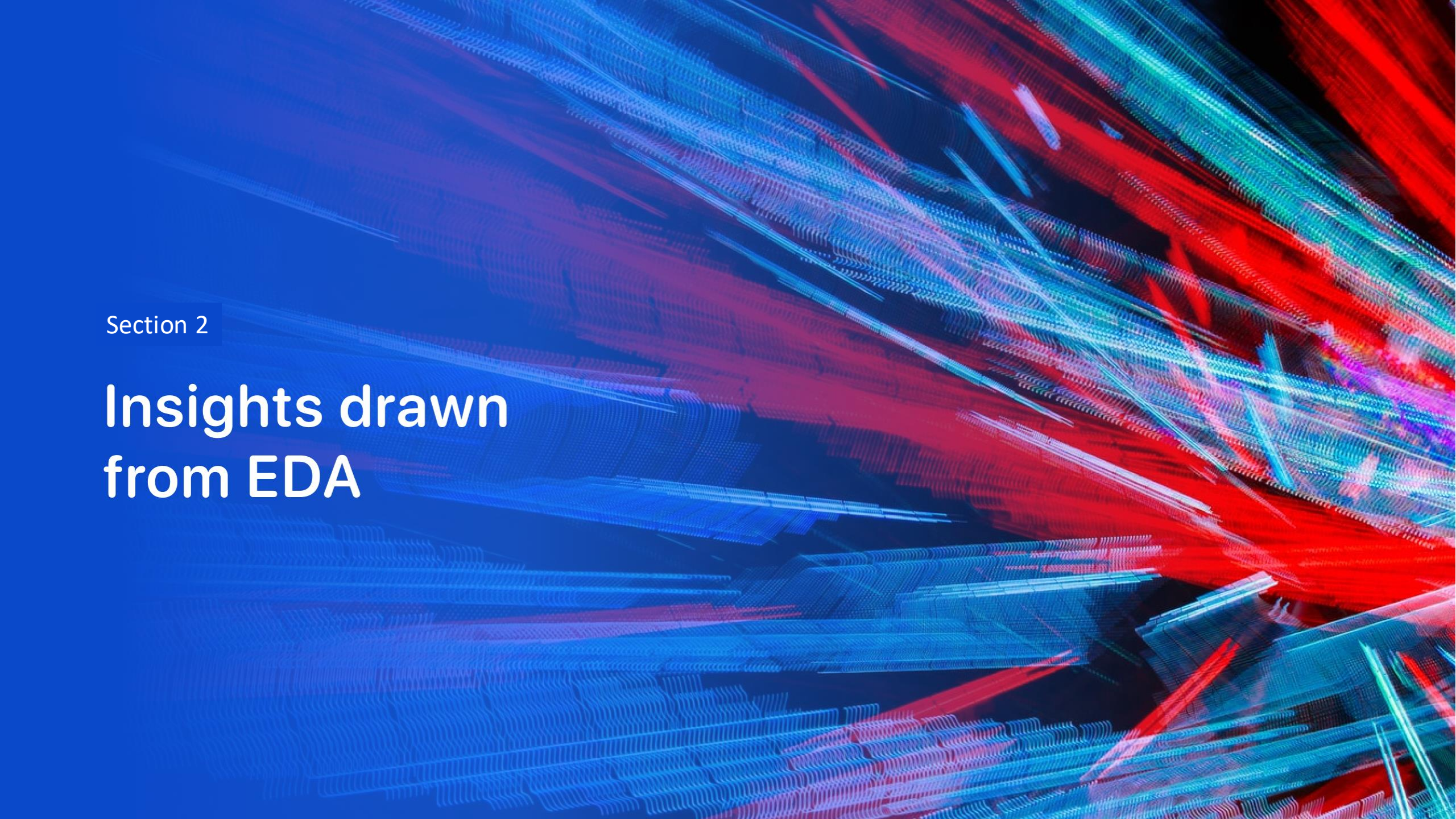
Results



Results

- Predictive analysis results

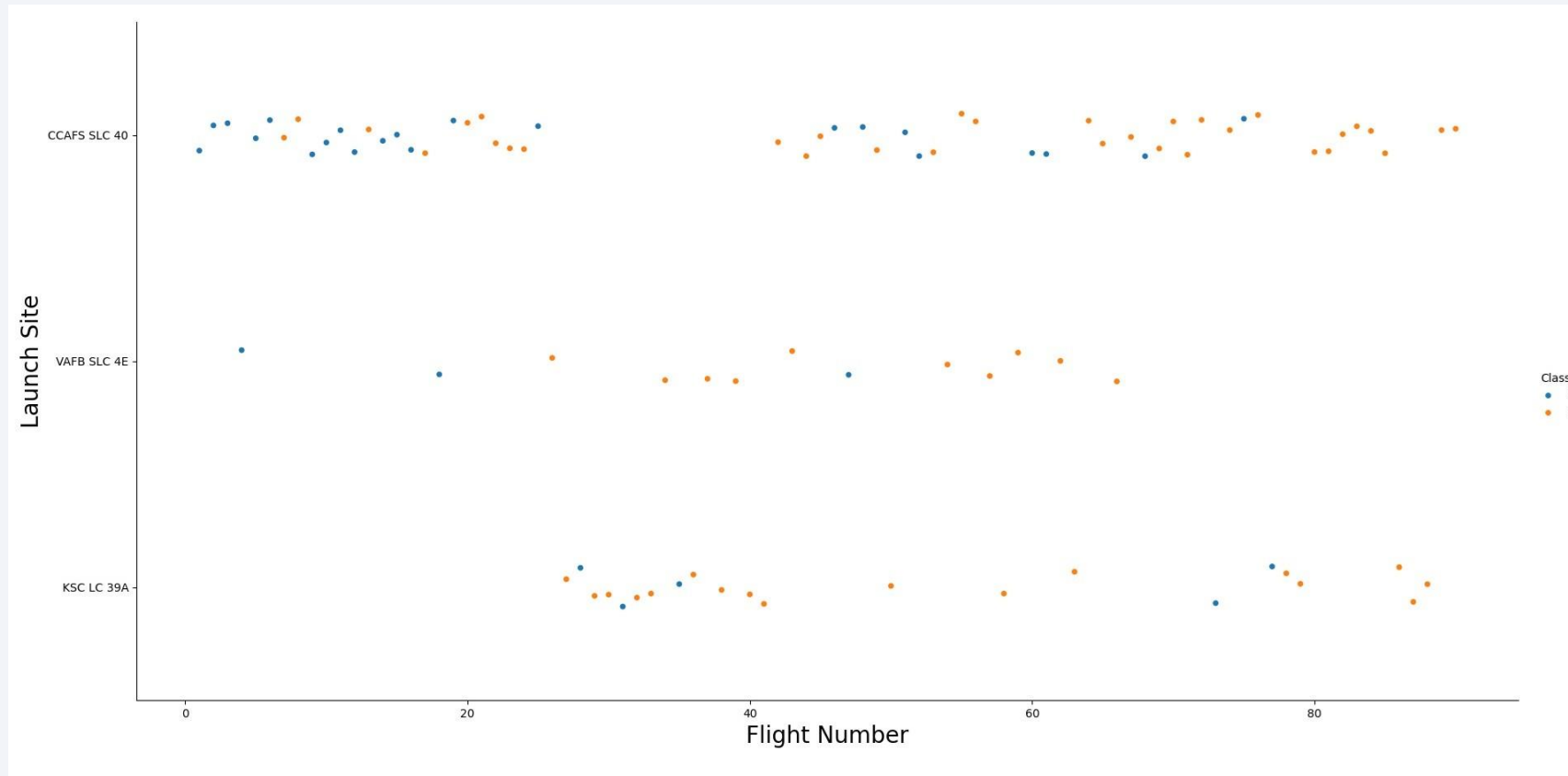
Method	Test Data Accuracy
Logistic Regression	0.833333
Support Vector Machine	0.833333
Decision Tree	0.833333
K-Nearest Neighbors	0.833333

The background of the slide is an abstract composition. It features a dark blue base color. Overlaid on this are numerous diagonal streaks in shades of blue and red, creating a sense of motion or data flow. A faint, light blue grid pattern is also visible, particularly in the lower-left quadrant. The overall effect is high-tech and digital.

Section 2

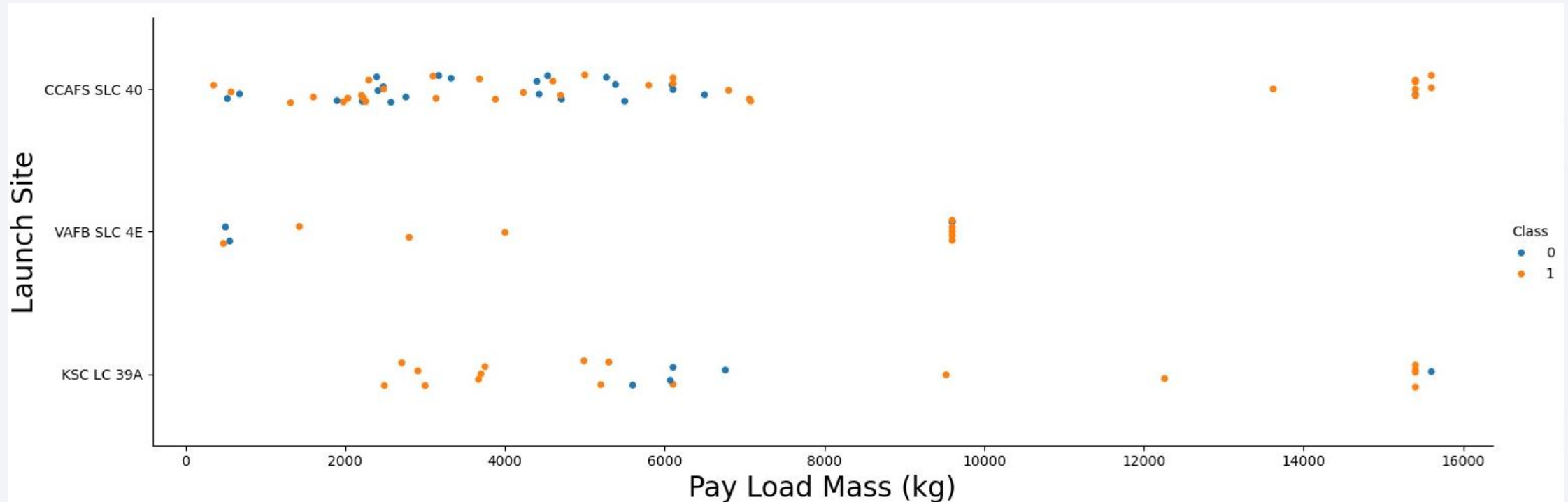
Insights drawn from EDA

Flight Number vs. Launch Site



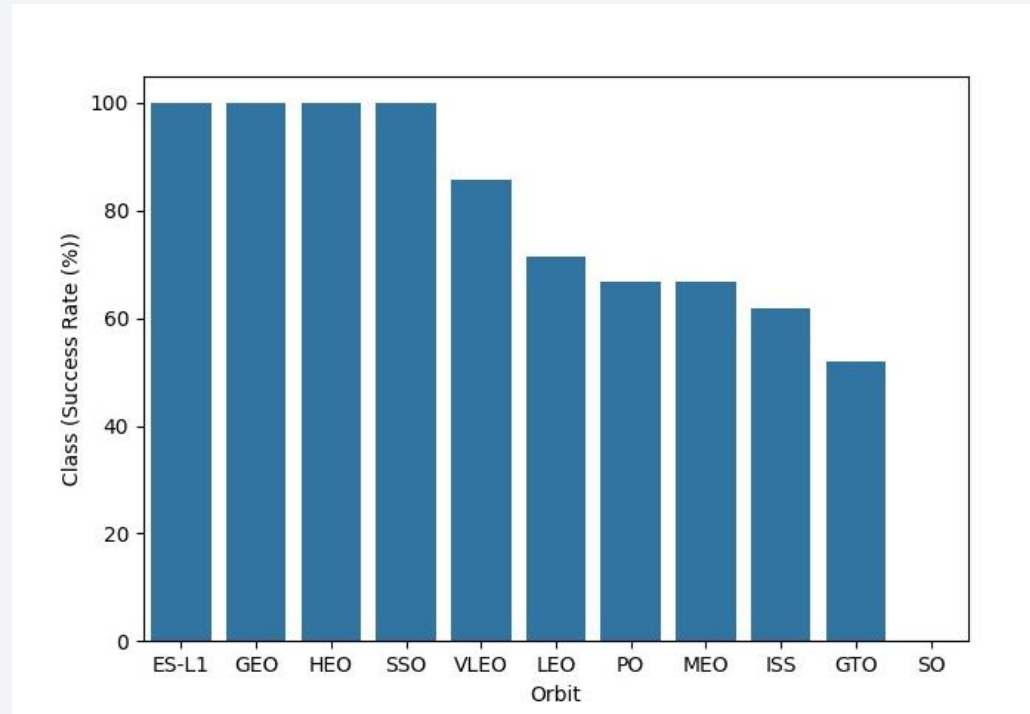
The graph shows the number of flights versus the launch sites. Here we can also see the success of landing of each launch site, '1' for successful and '0' for non successful. CCAFS SLC-40 is the most successful.

Payload vs. Launch Site



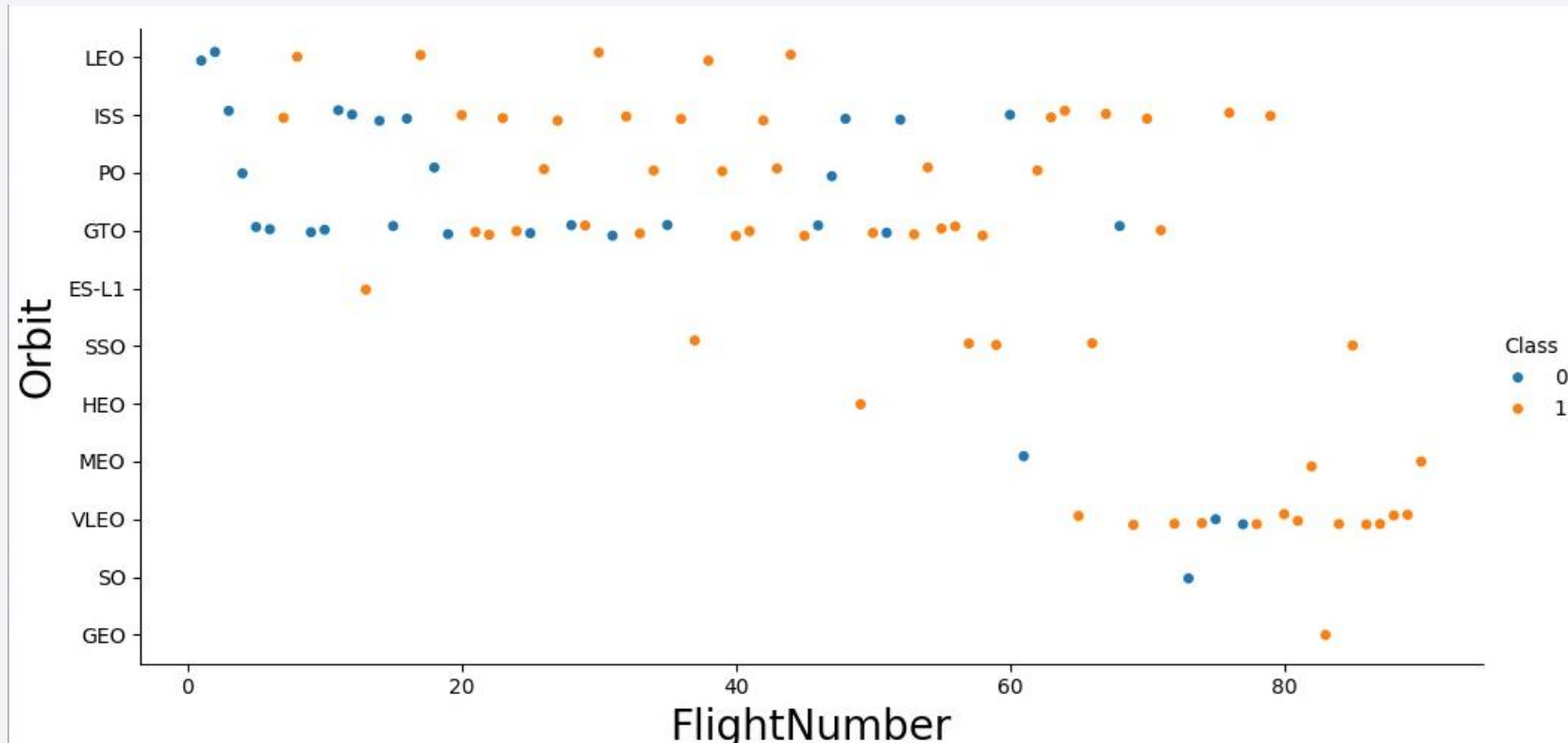
In Payload Mass vs. Launch Site scatter point chart we found for the VAFB SLC-4E launch site there are no rockets launches for heavy payload mass greater than 10000.

Success Rate vs. Orbit Type



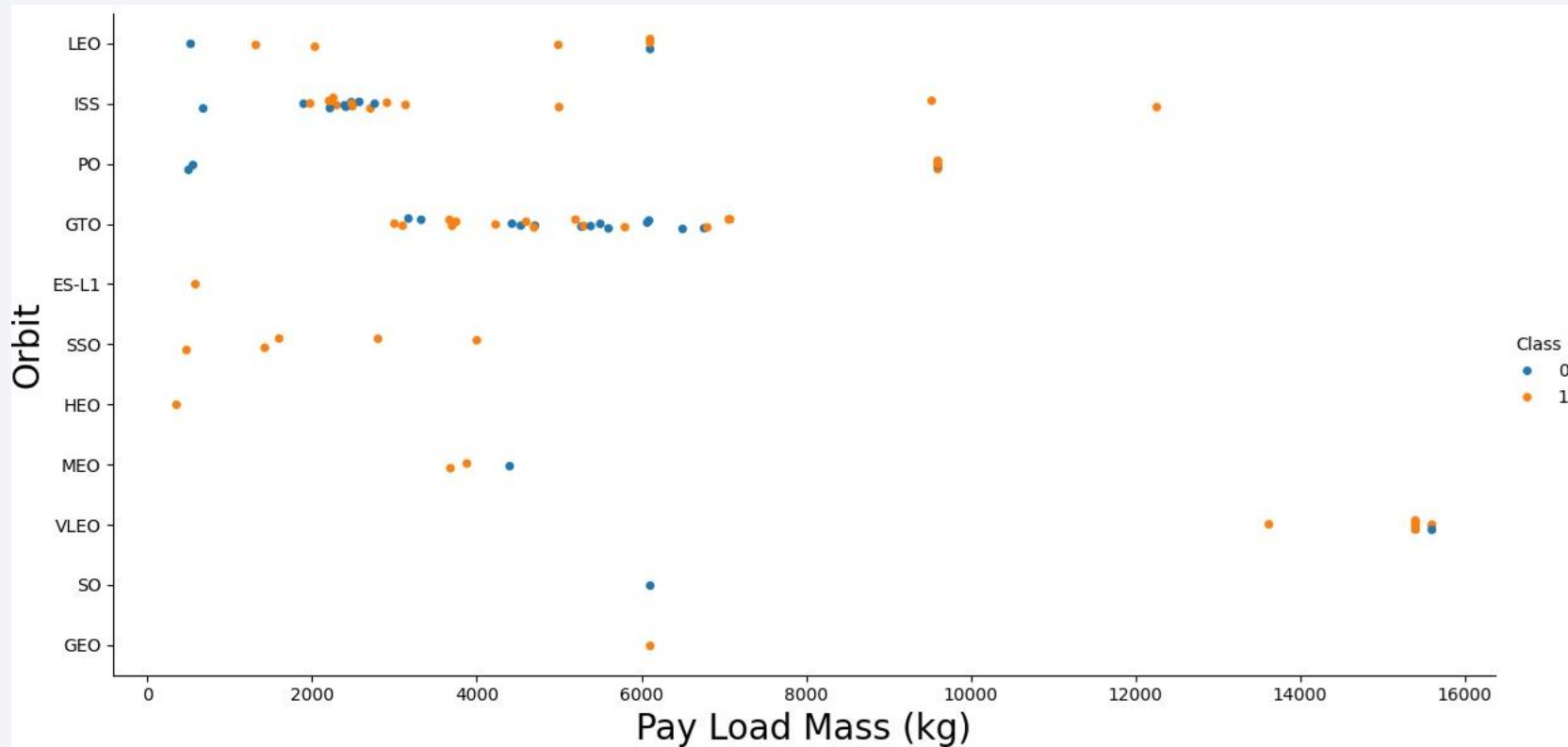
The highest success rate belongs to ES-L1, GEO, HEO and SSO orbits; the least success rate is for SO orbit.

Flight Number vs. Orbit Type



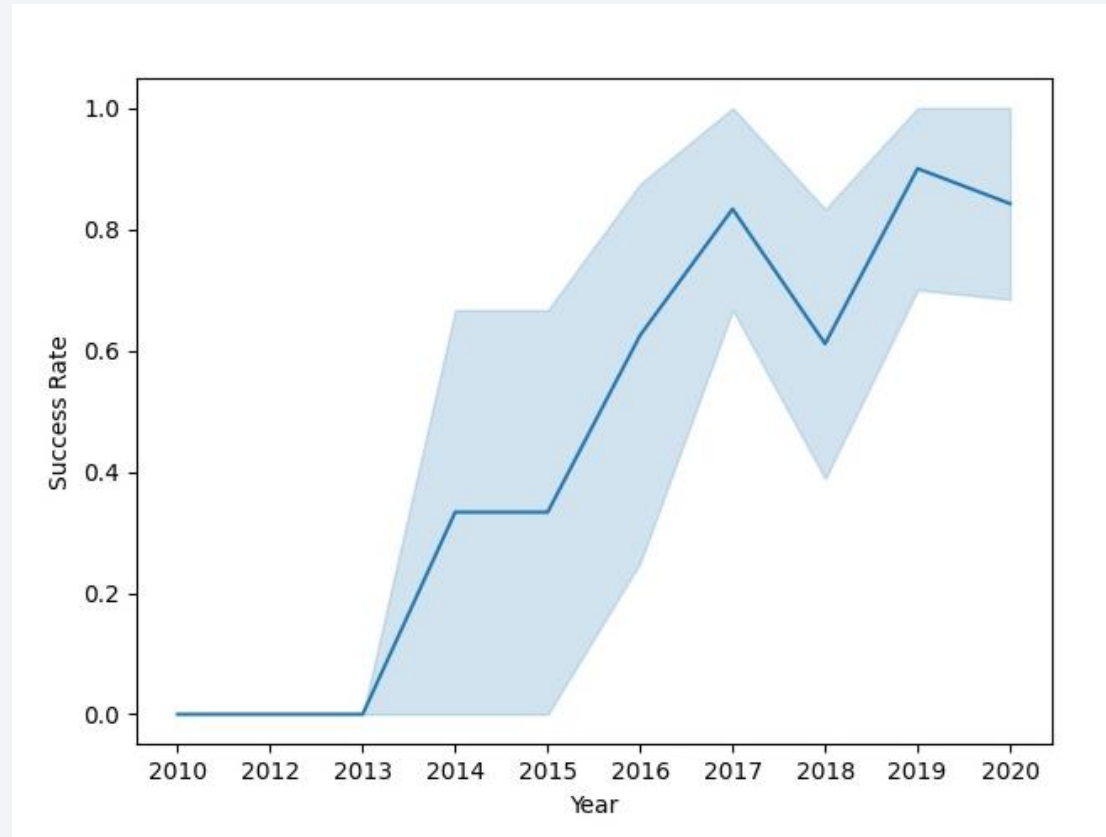
VLEO, LEO, ISS and POLAR are the orbits with more successful landing or positive landing rate. GTO it's difficult to distinguish between successful and unsuccessful landings as both outcomes are present.

Payload vs. Orbit Type



With heavy payloads the successful landing or positive landing rate are more for orbits PO, LEO, VLEO and ISS. However, for GTO, it's difficult to distinguish between successful and unsuccessful landings as both outcomes are present.

Launch Success Yearly Trend



The success rate since 2013 kept increasing till 2020.

All Launch Site Names

- Names of the unique launch sites

```
[12]: %sql select distinct(LAUNCH_SITE) from SPACEXTBL
      * sqlite:///my_data1.db
      Done.
[12]: Launch_Site
      CCAFS LC-40
      VAFB SLC-4E
      KSC LC-39A
      CCAFS SLC-40
```

Unique launch sites from 'SPACEXTBL' database using 'distinct' keyword to return only unique vales.

Launch Site Names Begin with 'CCA'

- 5 records where launch sites begin with 'CCA'

```
[13]: %sql select * from SPACEXTBL where LAUNCH_SITE like 'CCA%' limit 5
```

```
* sqlite:///my_data1.db  
Done.
```

```
[13]:
```

Date	Time (UTC)	Booster_Version	Launch_Site	Payload	PAYLOAD_MASS_KG_	Orbit	Customer	Mission_Outcome	Landing_Outcome
2010-06-04	18:45:00	F9 v1.0 B0003	CCAFS LC-40	Dragon Spacecraft Qualification Unit	0	LEO	SpaceX	Success	Failure (parachute)
2010-12-08	15:43:00	F9 v1.0 B0004	CCAFS LC-40	Dragon demo flight C1, two CubeSats, barrel of Brouere cheese	0	LEO (ISS)	NASA (COTS) NRO	Success	Failure (parachute)
2012-05-22	7:44:00	F9 v1.0 B0005	CCAFS LC-40	Dragon demo flight C2	525	LEO (ISS)	NASA (COTS)	Success	No attempt
2012-10-08	0:35:00	F9 v1.0 B0006	CCAFS LC-40	SpaceX CRS-1	500	LEO (ISS)	NASA (CRS)	Success	No attempt
2013-03-01	15:10:00	F9 v1.0 B0007	CCAFS LC-40	SpaceX CRS-2	677	LEO (ISS)	NASA (CRS)	Success	No attempt

We matched first letters 'CCA' from column 'LAUNCH_SITE' with 'like' keyword and 'limit 5' allow us to display only 5 records.

Total Payload Mass

- Total payload carried by boosters from NASA

```
[17]: %sql select sum(PAYLOAD_MASS_KG_) from SPACEXTBL where CUSTOMER = 'NASA (CRS)'  
      * sqlite:///my_data1.db  
      Done.  
[17]: sum(PAYLOAD_MASS_KG_)  
      45596
```

By using the function 'SUM' we summate the total of the column 'PAYLOAD_MASS_KG_' from the 'SPACEXTBL' database. Besides, we indicate that the boosters were launched by NASA with the 'CUSTOMER = 'NASA (CRS)'' query.

Average Payload Mass by F9 v1.1

- Average payload mass carried by booster version F9 v1.1

```
[18]: %sql select avg(PAYLOAD_MASS_KG_) from SPACEXTBL where BOOSTER_VERSION = 'F9 v1.1'
* sqlite:///my_data1.db
Done.
[18]: avg(PAYLOAD_MASS_KG_)
2928.4
```

The 'avg' function applied to the column 'PAYLOAD_MASS_KG_' from the 'SPACEXTBL' database gives us the average of payload mass. We filtered the booster version required with the 'where' keyword.

First Successful Ground Landing Date

- Date of the first successful landing outcome on ground pad

```
[20]: %sql select min(DATE) from SPACEXTBL where LANDING_OUTCOME = 'Success (ground pad)'  
      * sqlite:///my_data1.db  
Done.  
[20]: min(DATE)  
      2015-12-22
```

By using 'min' on the column 'DATE' and filtering the success on ground in column 'LANDING_OUTCOME' with 'where' keyword we retrieved the first date required.

Successful Drone Ship Landing with Payload between 4000 and 6000

- Names of boosters which have successfully landed on drone ship and had payload mass greater than 4000 but less than 6000

```
[21]: %sql select BOOSTER_VERSION from SPACEXTBL where LANDING_OUTCOME = 'Success (drone ship)' and PAYLOAD_MASS_KG_ > 4000 and PAYLOAD_MASS_KG_ < 6000
* sqlite:///my_data1.db
Done.
[21]: Booster_Version
```

F9 FT B1022
F9 FT B1026
F9 FT B1021.2
F9 FT B1031.2

We query the 'BOOSTER_VERSION' from the 'SPACEXTBL' database while filtering 'LANDING_OUTCOME' in value 'Success (drone ship)' and the range between 4000 and 6000 on 'PAYLOAD_MASS_KG_'.

Total Number of Successful and Failure Mission Outcomes

- Total number of successful and failure mission outcomes

```
%sql select count(MISSION_OUTCOME) from SPACEXTBL where (MISSION_OUTCOME =  
'Success' or MISSION_OUTCOME = 'Failure (in flight)' or MISSION_OUTCOME = 'Success  
(payload status unclear)' or MISSION_OUTCOME = 'Success `)
```

```
[35]: %sql select count(MISSION_OUTCOME) from SPACEXTBL where (MISSION_OUTCOME = 'Success' or MISSION_OUTCOME = 'Failure (in flight)' or MISSION_OUTCOME = 'Success  
      * sqlite:///my_data1.db  
      Done.  
[35]: count(MISSION_OUTCOME)  
      101
```

The 'count' function returns the number of rows that matches the specified criterion: in column 'MISSION_OUTCOME' we search for 'Success', 'Failure (in flight)' and 'Success (payload status unclear)'.

Boosters Carried Maximum Payload

- Names of the booster which have carried the maximum payload mass

```
[38]: %sql select BOOSTER_VERSION from SPACEXTBL where PAYLOAD_MASS_KG_ = (select max(PAYLOAD_MASS_KG_) from SPACEXTBL)

* sqlite:///my_data1.db
Done.
[38]: Booster_Version
```

F9 B5 B1048.4
F9 B5 B1049.4
F9 B5 B1051.3
F9 B5 B1056.4
F9 B5 B1048.5
F9 B5 B1051.4
F9 B5 B1049.5
F9 B5 B1060.2
F9 B5 B1058.3
F9 B5 B1051.6
F9 B5 B1060.3
F9 B5 B1049.7

We selected from column 'BOOSTER_VERSION' the records that matched the maximum value (max function) from the column 'PAYLOAD_MASS_KG_' in the database.

2015 Launch Records

- List of the failed landing_outcomes in drone ship, their booster versions, and launch site names for in year 2015

```
%sql select SUBSTR(Date, 6, 2), BOOSTER_VERSION, LAUNCH_SITE, LANDING_OUTCOME
from SPACEXTBL where (LANDING_OUTCOME = 'Failure (drone ship)') and
SUBSTR(Date, 0, 5) = '2015'
```

```
[42]: %sql select SUBSTR(Date, 6, 2), BOOSTER_VERSION, LAUNCH_SITE, LANDING_OUTCOME from SPACEXTBL where (LANDING_OUTCOME = 'Failure (drone ship)') and SUBSTR(Date, 0, 5) = '2015'
* sqlite:///my_data1.db
Done.
```

[42]:	SUBSTR(Date, 6, 2)	Booster_Version	Launch_Site	Landing_Outcome
	01	F9 v1.1 B1012	CCAFS LC-40	Failure (drone ship)
	04	F9 v1.1 B1015	CCAFS LC-40	Failure (drone ship)

We selected records for months, 'BOSTER_VERSION', 'LAUNCH_SITE', 'LANDING_OUTCOME' where 'LANDING_OUTCOME' is listed as 'Failure (drone ship)' and year as '2015'. SQLite does not support month names, so we needed to use substr(Date, 6,2) as month to get the months and substr(Date,0,5)='2015' for year.

Rank Landing Outcomes Between 2010-06-04 and 2017-03-20

- Rank of landing outcomes (such as Failure (drone ship) or Success (ground pad)) between the date 2010-06-04 and 2017-03-20, in descending order

```
[57]: %sql select * from SPACE_TBL where (DATE between '2010-06-04' and '2017-03-20') ORDER BY DATE DESC
```

```
* sqlite:///my_data1.db
Done.
```

```
[57]:
```

Date	Time (UTC)	Booster_Version	Launch_Site	Payload	PAYLOAD_MASS_KG	Orbit	Customer	Mission_Outcome	Landing_Outcome
2017-03-16	6:00:00	F9 FT B1030	KSC LC-39A	EchoStar 23	5600	GTO	EchoStar	Success	No attempt
2017-02-19	14:39:00	F9 FT B1031.1	KSC LC-39A	SpaceX CRS-10	2490	LEO (ISS)	NASA (CRS)	Success	Success (ground pad)
2017-01-14	17:54:00	F9 FT B1029.1	VAFB SLC-4E	Iridium NEXT 1	9600	Polar LEO	Iridium Communications	Success	Success (drone ship)
2016-08-14	5:26:00	F9 FT B1026	CCAFS LC-40	JCSAT-16	4600	GTO	SKY Perfect JSAT Group	Success	Success (drone ship)
2016-07-18	4:45:00	F9 FT B1025.1	CCAFS LC-40	SpaceX CRS-9	2257	LEO (ISS)	NASA (CRS)	Success	Success (ground pad)
2016-06-15	14:29:00	F9 FT B1024	CCAFS LC-40	ABS-2A Eutelsat 117 West B	3600	GTO	ABS Eutelsat	Success	Failure (drone ship)
2016-05-27	21:39:00	F9 FT B1023.1	CCAFS LC-40	Thaicom 8	3100	GTO	Thaicom	Success	Success (drone ship)
2016-05-06	5:21:00	F9 FT B1022	CCAFS LC-40	JCSAT-14	4696	GTO	SKY Perfect JSAT Group	Success	Success (drone ship)
2016-04-08	20:43:00	F9 FT B1021.1	CCAFS LC-40	SpaceX CRS-8	3136	LEO (ISS)	NASA (CRS)	Success	Success (drone ship)
2016-03-04	23:35:00	F9 FT B1020	CCAFS LC-40	SES-9	5271	GTO	SES	Success	Failure (drone ship)
2016-01-17	18:42:00	F9 v1.1 B1017	VAFB SLC-4E	Jason-3	553	LEO	NASA (LSP) NOAA CNES	Success	Failure (drone ship)
2015-12-22	1:29:00	F9 FT B1019	CCAFS LC-40	OG2 Mission 2 11 Orbcomm-OG2 satellites	2034	LEO	Orbcomm	Success	Success (ground pad)
2015-06-28	14:21:00	F9 v1.1 B1018	CCAFS LC-40	SpaceX CRS-7	1952	LEO (ISS)	NASA (CRS)	Failure (in flight)	Precluded (drone ship)

Rank Landing Outcomes Between 2010-06-04 and 2017-03-20

2015-04-27	23:03:00	F9 v1.1 B1016	CCAFS LC-40	Turkmen 52 / MonacoSAT	4707	GTO	Turkmenistan National Space Agency	Success	No attempt
2015-04-14	20:10:00	F9 v1.1 B1015	CCAFS LC-40	SpaceX CRS-6	1898	LEO (ISS)	NASA (CRS)	Success	Failure (drone ship)
2015-03-02	3:50:00	F9 v1.1 B1014	CCAFS LC-40	ABS-3A Eutelsat 115 West B	4159	GTO	ABS Eutelsat	Success	No attempt
2015-02-11	23:03:00	F9 v1.1 B1013	CCAFS LC-40	DSCOVR	570	HEO	U.S. Air Force NASA NOAA	Success	Controlled (ocean)
2015-01-10	9:47:00	F9 v1.1 B1012	CCAFS LC-40	SpaceX CRS-5	2395	LEO (ISS)	NASA (CRS)	Success	Failure (drone ship)
2014-09-21	5:52:00	F9 v1.1 B1010	CCAFS LC-40	SpaceX CRS-4	2216	LEO (ISS)	NASA (CRS)	Success	Uncontrolled (ocean)
2014-09-07	5:00:00	F9 v1.1 B1011	CCAFS LC-40	AsiaSat 6	4428	GTO	AsiaSat	Success	No attempt
2014-08-05	8:00:00	F9 v1.1	CCAFS LC-40	AsiaSat 8	4535	GTO	AsiaSat	Success	No attempt
2014-07-14	15:15:00	F9 v1.1	CCAFS LC-40	OG2 Mission 1 6 Orbcomm-OG2 satellites	1316	LEO	Orbcomm	Success	Controlled (ocean)
2014-04-18	19:25:00	F9 v1.1	CCAFS LC-40	SpaceX CRS-3	2296	LEO (ISS)	NASA (CRS)	Success	Controlled (ocean)
2014-01-06	22:06:00	F9 v1.1	CCAFS LC-40	Thaicom 6	3325	GTO	Thaicom	Success	No attempt
2013-12-03	22:41:00	F9 v1.1	CCAFS LC-40	SES-8	3170	GTO	SES	Success	No attempt
2013-09-29	16:00:00	F9 v1.1 B1003	VAFB SLC-4E	CASSIOPE	500	Polar LEO	MDA	Success	Uncontrolled (ocean)
2013-03-01	15:10:00	F9 v1.0 B0007	CCAFS LC-40	SpaceX CRS-2	677	LEO (ISS)	NASA (CRS)	Success	No attempt
2012-10-08	0:35:00	F9 v1.0 B0006	CCAFS LC-40	SpaceX CRS-1	500	LEO (ISS)	NASA (CRS)	Success	No attempt
2012-05-22	7:44:00	F9 v1.0 B0005	CCAFS LC-40	Dragon demo flight C2	525	LEO (ISS)	NASA (COTS)	Success	No attempt

Rank Landing Outcomes Between 2010-06-04 and 2017-03-20

2010-12-08	15:43:00	F9 v1.0 B0004	CCAFS LC-40	Dragon demo flight C1, two CubeSats, barrel of Brouere cheese	0	LEO (ISS)	NASA (COTS) NRO	Success	Failure (parachute)
2010-06-04	18:45:00	F9 v1.0 B0003	CCAFS LC-40	Dragon Spacecraft Qualification Unit	0	LEO	SpaceX	Success	Failure (parachute)

We selected records from database filtering on 'DATE' in the required dates. We displayed the records in descending order with the query 'ORDER BY DATE DESC'.

A satellite view of Earth from space, showing the curvature of the planet and city lights at night. The image is a composite of a solid blue background on the left and a satellite photograph of Earth on the right. The Earth's surface is dark, with numerous bright yellow and orange lights representing cities and urban areas. The horizon of the Earth is visible as a thin, curved line separating the dark surface from the deep blue of space.

Section 3

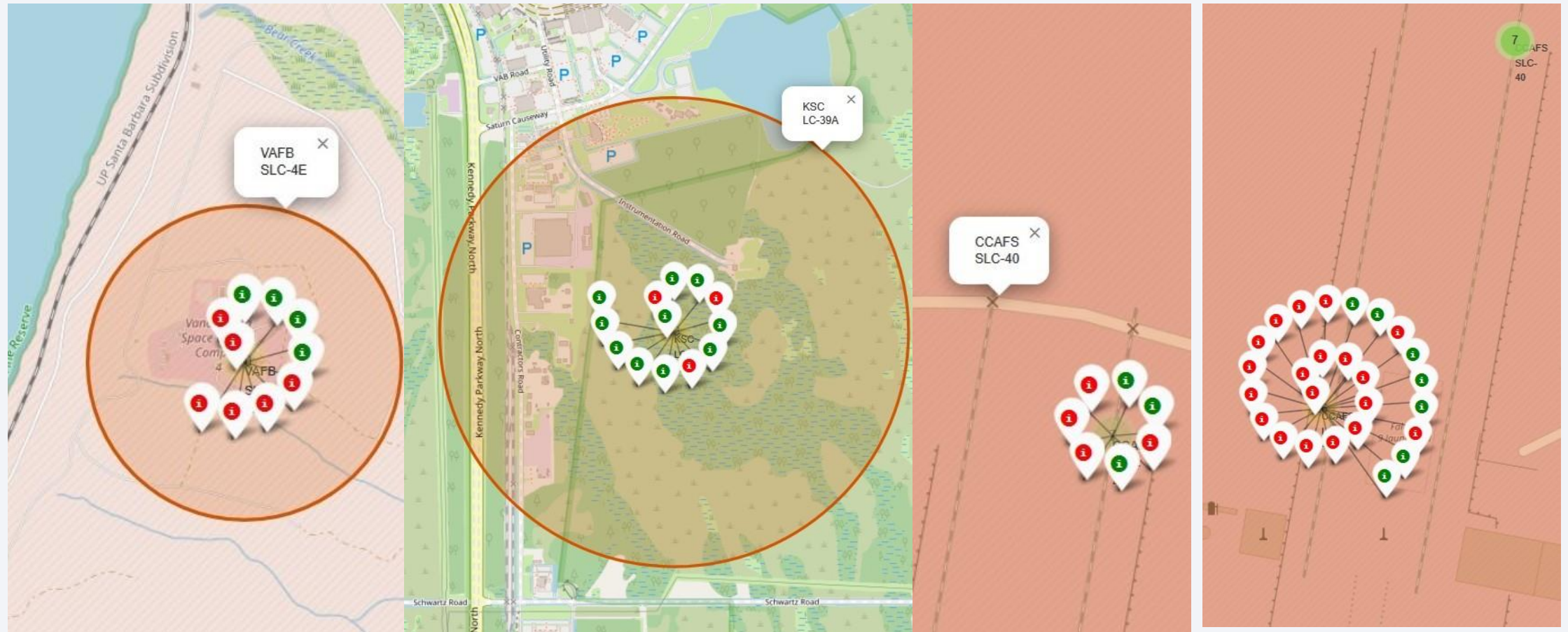
Launch Sites Proximities Analysis

All Launch Sites Global Map



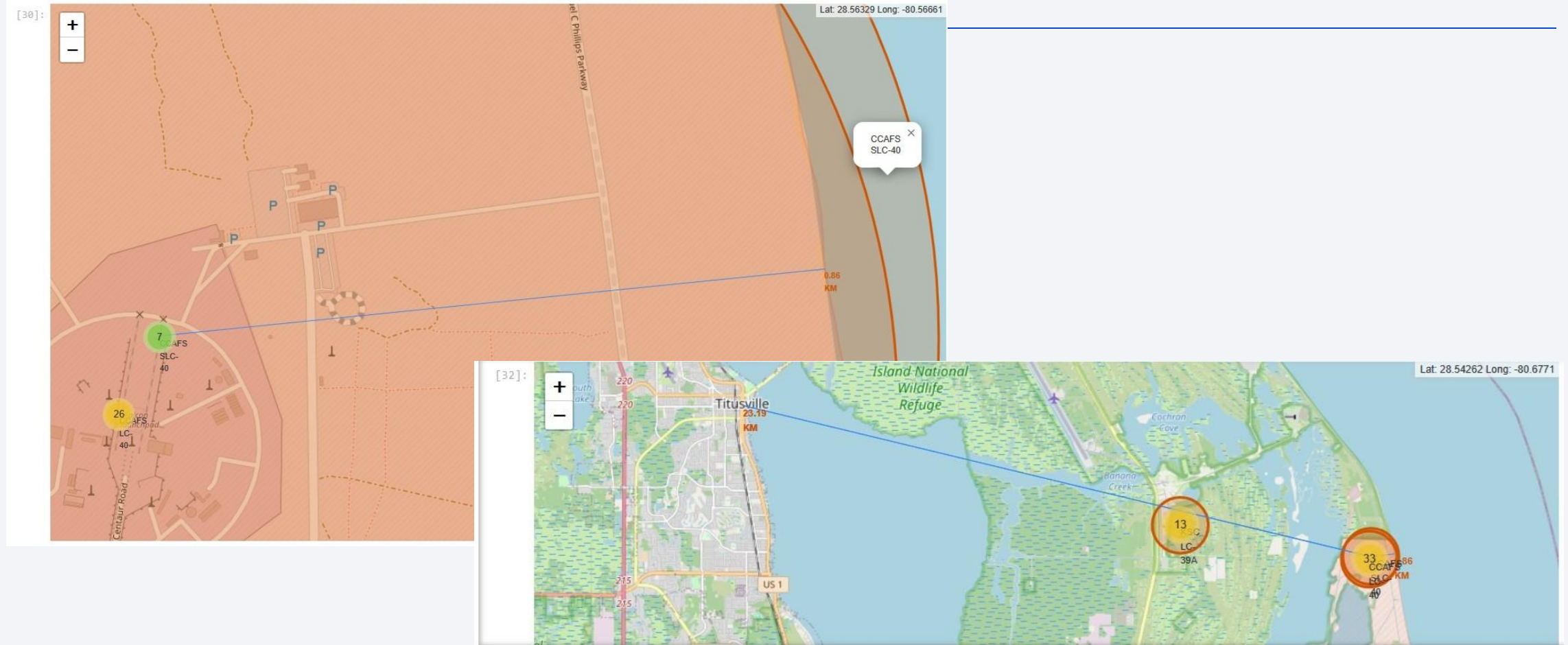
SpaceX launch sites are in Florida and California (USA).

Color Labeled Launch Outcomes

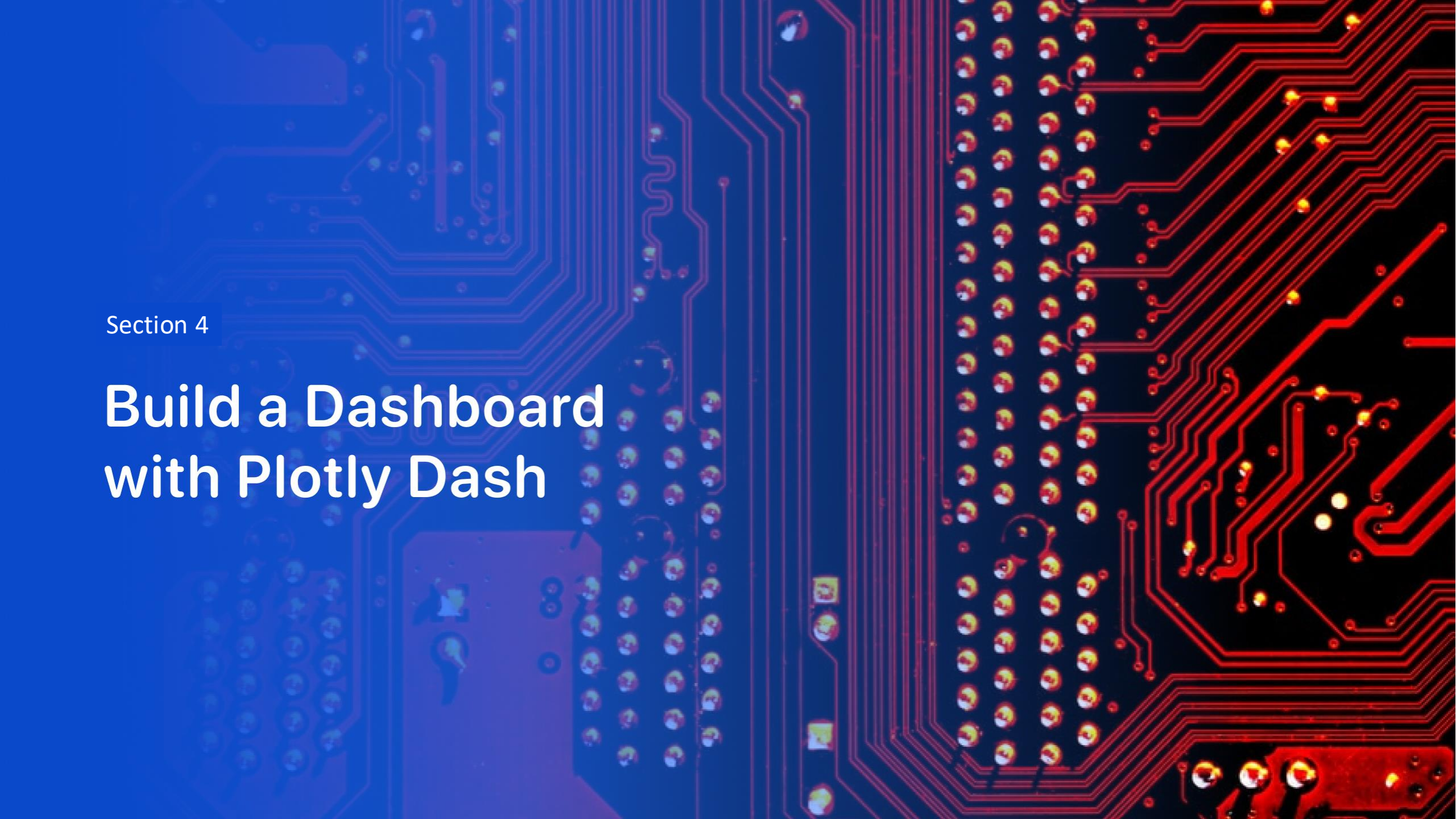


Green Marker indicates successful launches and Red Marker indicates failed launches.

Selected proximities to a Launch Site



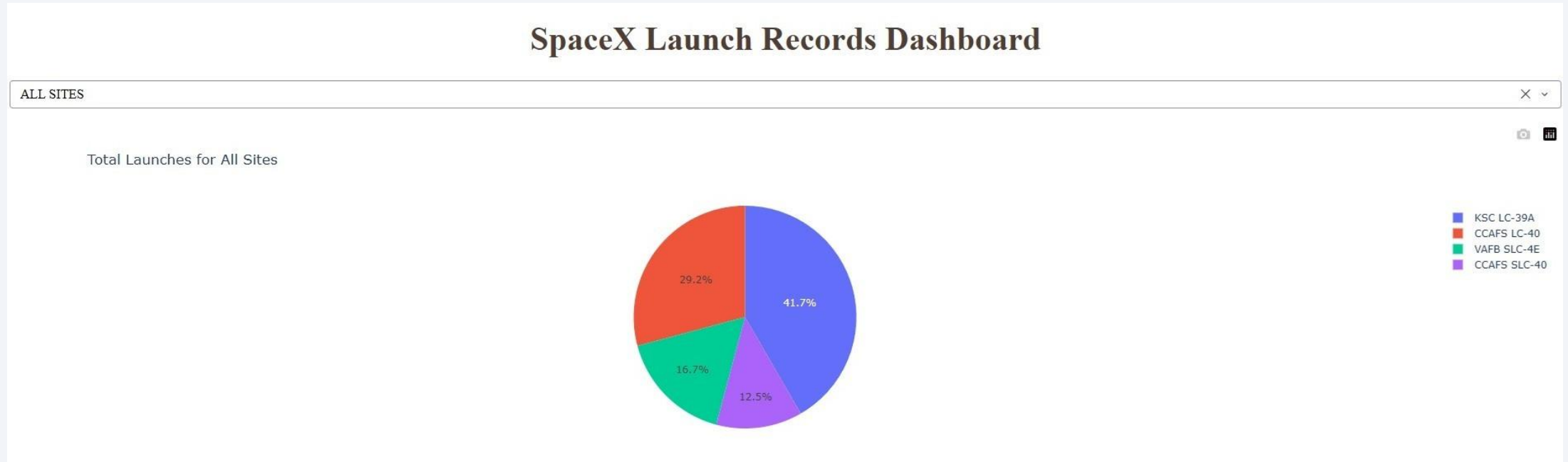
First map shows the shortest distance from site CCAFS SLC-40 to the coastline (0.86 km). Second city shows the closest city from site CCAFS SLS-40 which is Titusville, Florida (23.19 km).



Section 4

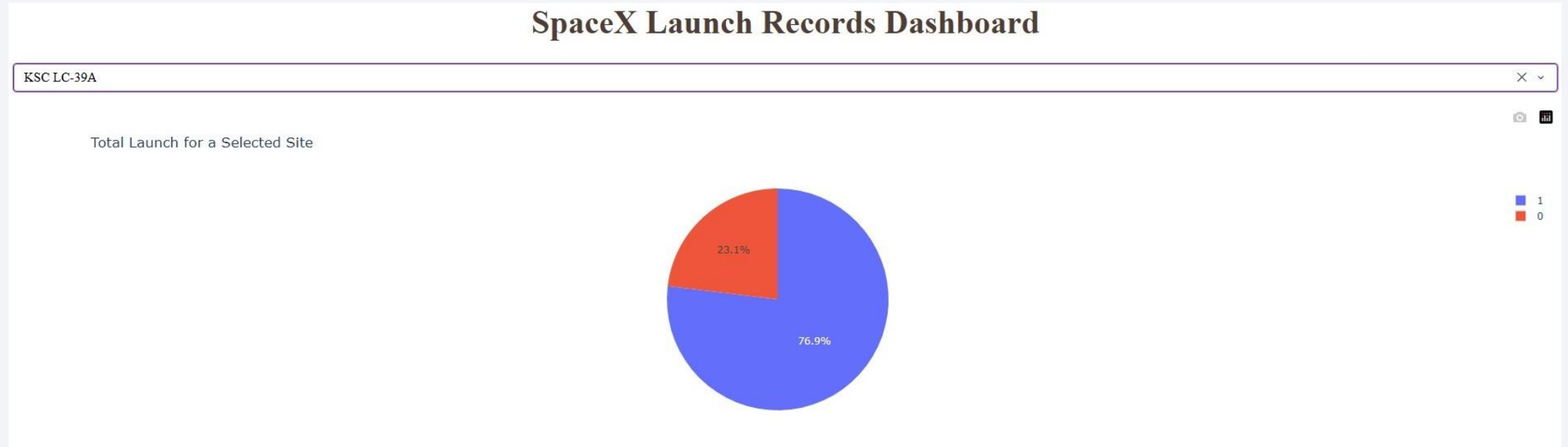
Build a Dashboard with Plotly Dash

Launch success for all sites



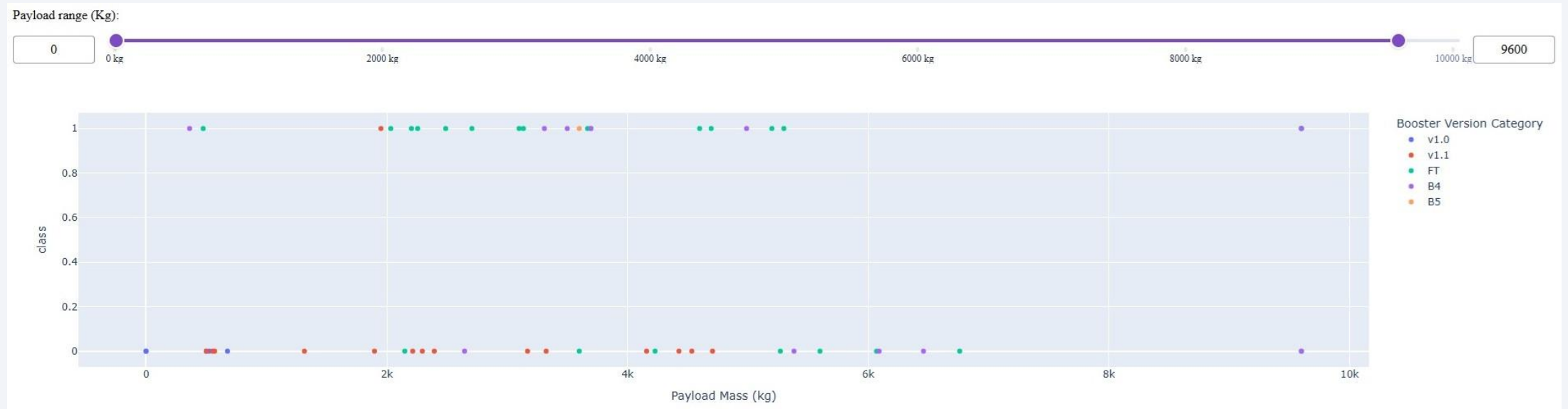
We can see that KSC LC-39A had the highest rate of success and CCAFS SLC-40 the lowest.

Launch site with highest rate success



We can see that KSC LC-39A had 76.9% of successful launches.

Payload vs Launch Outcome by Booster Version



The FT booster version had the most successful rate of launches. The B4 booster version made the heaviest payload outcome.



Section 5

Predictive Analysis (Classification)

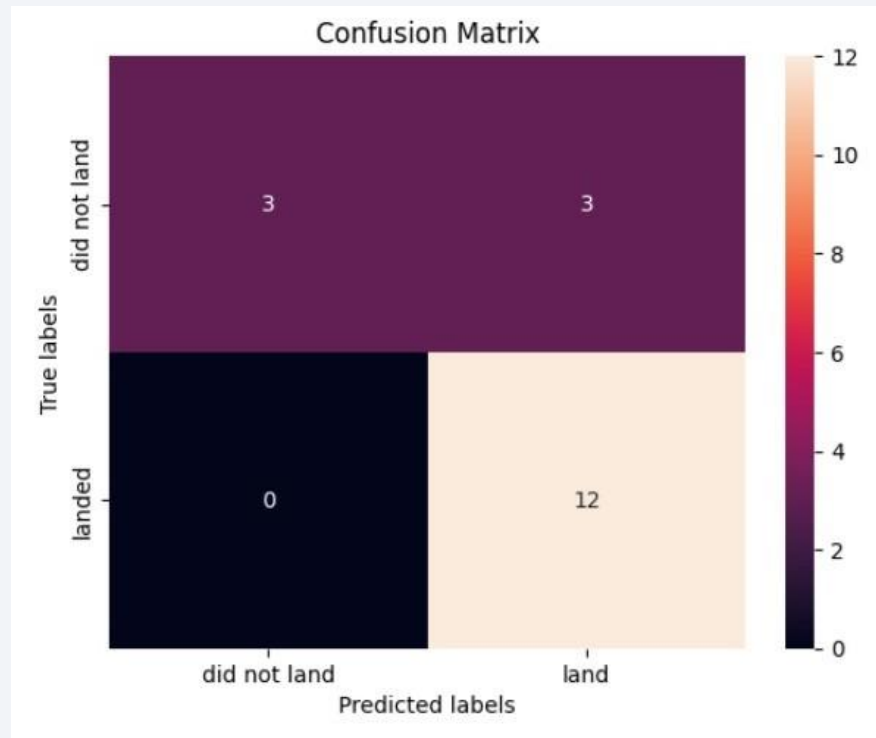
Classification Accuracy

[48]: 0

Method	Test Data Accuracy
Logistic Regression	0.833333
Support Vector Machine	0.833333
Decision Tree	0.833333
K-Nearest Neighbors	0.833333

With 83.33% of accuracy on the test data all models are equally optional.

Confusion Matrix



We had 12 True Positives, 3 False Positives, 3 True Negatives and 0 False Negatives.

Conclusions

- Linear Regression, Support Vector Machine, Decision Tree and K-Nearest Neighbor are equally eligible for this dataset.
- The success of SpaceX launches has been improving since 2013.
- Orbits ES-L1, GEO, HEO and SSO had the highest rate of success.
- KSC LC-39A had the most successful launches of all sites with 76.9% of success.
- The B4 booster version has made the heaviest payload outcome.

Appendix

- **Obtaining the number of launches for each site.**

```
# Use the method 'value_counts()' on the column 'LaunchSite' to determine the number of launches on each site:  
  
# Apply value_counts() on column LaunchSite  
df["LaunchSite"].value_counts()
```

- **Calculating the number and occurrence of each orbit.**

```
# Use the method .value_counts() to determine the number and occurrence of each orbit in the column Orbit  
  
# Apply value_counts on Orbit column  
df["Orbit"].value_counts()  
df["Orbit"].value_counts("Orbit")
```

- **Features used in success prediction in EDA visualization.**

```
features = df[['FlightNumber', 'PayloadMass', 'Orbit', 'LaunchSite', 'Flights',  
              'GridFins', 'Reused', 'Legs', 'LandingPad', 'Block', 'ReusedCount', 'Serial']]
```

Appendix

- Creating a folium 'Map' object, with an initial center location to be NASA Johnson Space Center at Houston, Texas.

```
# Start location is NASA Johnson Space Center
nasa_coordinate = [29.559684888503615, -95.0830971930759]
site_map = folium.Map(location=nasa_coordinate, zoom_start=10)
```

- This function is to plot the confusion matrix:

```
def plot_confusion_matrix(y, y_predict):
    "this function plots the confusion matrix"
    from sklearn.metrics import confusion_matrix

    cm = confusion_matrix(y, y_predict)
    ax= plt.subplot()
    sns.heatmap(cm, annot=True, ax = ax); #annot=True to annotate cells
    ax.set_xlabel('Predicted labels')
    ax.set_ylabel('True labels')
    ax.set_title('Confusion Matrix');
    ax.xaxis.set_ticklabels(['did not land', 'land']); ax.yaxis.set_ticklabels(['did not land', 'landed'])
    plt.show()
```

Thank you!

